MAT2540, Classwork9, Spring2026

8.1 Applications of Recurrence Relations (Conti.)

8. Example of Dynamic programming: Dynamic Programming Algorithm for Scheduling Talks.

Suppose we have a group of proposed talks with preset start and end times. Devise a dynamic programming algorithm to have the largest possible combined attendance of the scheduled talks, under the assumptions that once a talk starts, it continues until it ends, no two talks can proceed at the same time, and a talk can begin at the same time another one ends. Assume that we have n talks, talk j begins at time s_j (where s stands for start) and ends at time e_j (where e stands for end) for $j = 1, \dots, n$.

Algorithm:

Suppose the talk j will be attended by w_j students. We want a schedule that maximizes the total number of student attendees.

We denote by $T(j)$ the maximum number of total attendees for an optimal schedule from the first j talks, so $T(n)$ is the maximal number of total attendees for an optimal schedule for all n talks.

We first sort the talks in order of increasing end time. It means $e_1 \le e_2 \le e_3 \le \cdots \le e_n$

(i) Talk j belongs to the optimal schedule: $p(j) = i$, $(i \ne 0,\ i < j)$

① The takes $p(j)+1$, $p(j)+2, \dots, j-1$ do not belong to optimal schedule

② From Talk 1 to Talk $p(j)$, we have $T(p(j))$ of students

③ We have w_j students in Talk j

These imply that $T(j) = w_j + T(p(j))$

(ii) Talk j doesn't belongs to the optimal schedule $\rightarrow T(j)$

The optimal schedule from Talk 1 to Talk j is the same as the $T(j-1)$ optimal schedule from Talk 1 to Talk $j-1$ $\Rightarrow T(j) = T(j-1)$

Pseudocode:

$j = 1, \dots, n$

procedure *Maximum Attendees*(s_j (start time), e_j (end time), w_j (# of attendee))

sort talks by end time and relabel so that $e_1 \le e_2 \le e_3 \le \cdots \le e_n$

for $j := 1$ **to** n

  if no talk i with $i < j$ is compatible with talk j.

    $p(j) = 0$

  else $p(j) := \max\{i\}$ where $i < j$ and talk i is compatible with Talk j

  $T(0) := 0$

for $j := 1$ **to** n

  $T(j) := \max(w_j + T(p(j)),\ T(j-1))$

return $T(n)$ { $T(n)$ is maximum number of attendees }

memoization

8.2 Solving Linear Recurrence Relations

1. Let H_n denote the number of moves needed to solve the Tower of Hanoi puzzle with n disks. The recurrence relation for the sequence $\{H_n\}$ is given by the initial condition $H_1 = 1$ and the rule $H_n = 2H_{n-1} + 1$. Find an explicit formula for H_n.

2. Definition: Linear Homogeneous Recurrence Relation of degree k

   A linear homogeneous recurrence relation of degree k with constant coefficients is a recurrence relation of the form

   $$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}, \text{ where } c_1, c_2, \ldots, c_k \text{ are real numbers, and } c_k \neq 0.$$

   Linear:

   Homogeneous:

   The coefficients:

   Degree:

3. Examples of the Linear Homogeneous Recurrence Relation.

   a) $P_n = (1.11)P_{n-1}$:

   b) $f_n = f_{n-1} + f_{n-2}$:

   c) $a_n = a_{n-5}$:

   d) $a_n = a_{n-1} + (a_{n-2})^2$:

   e) $H_n = 2H_{n-1} + 1$:

   f) $B_n = nB_{n-1}$:

4. Characteristic equation and Characteristic roots

   For linear homogeneous recurrence relation with constant coefficients, we can use two key ideas to find all their solutions.

   1) These recurrence relations have solutions of the form $a_n = r^n$, where r is a constant:

      Observe that $a_n = r^n$ is a solution of the recurrence relation

      $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$ if and only if _____

      Then, when $r \neq 0$, we can simplify this equation into

      _____

      We call this the _____ of the recurrence relation. The solutions of this equation are called the _____ of the recurrence relation.

   2) A linear combination of two solutions of a linear homogeneous recurrence relation is also a solution:

      suppose that s_n and t_n are both solutions of $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$. Then

      $s_n =$ _____ and $t_n =$ _____

      Now suppose that b_1 and b_2 are real numbers. Then

      $b_1 s_n + b_2 t_n =$ _____